Inverse Kinematics Design

1. Content Description

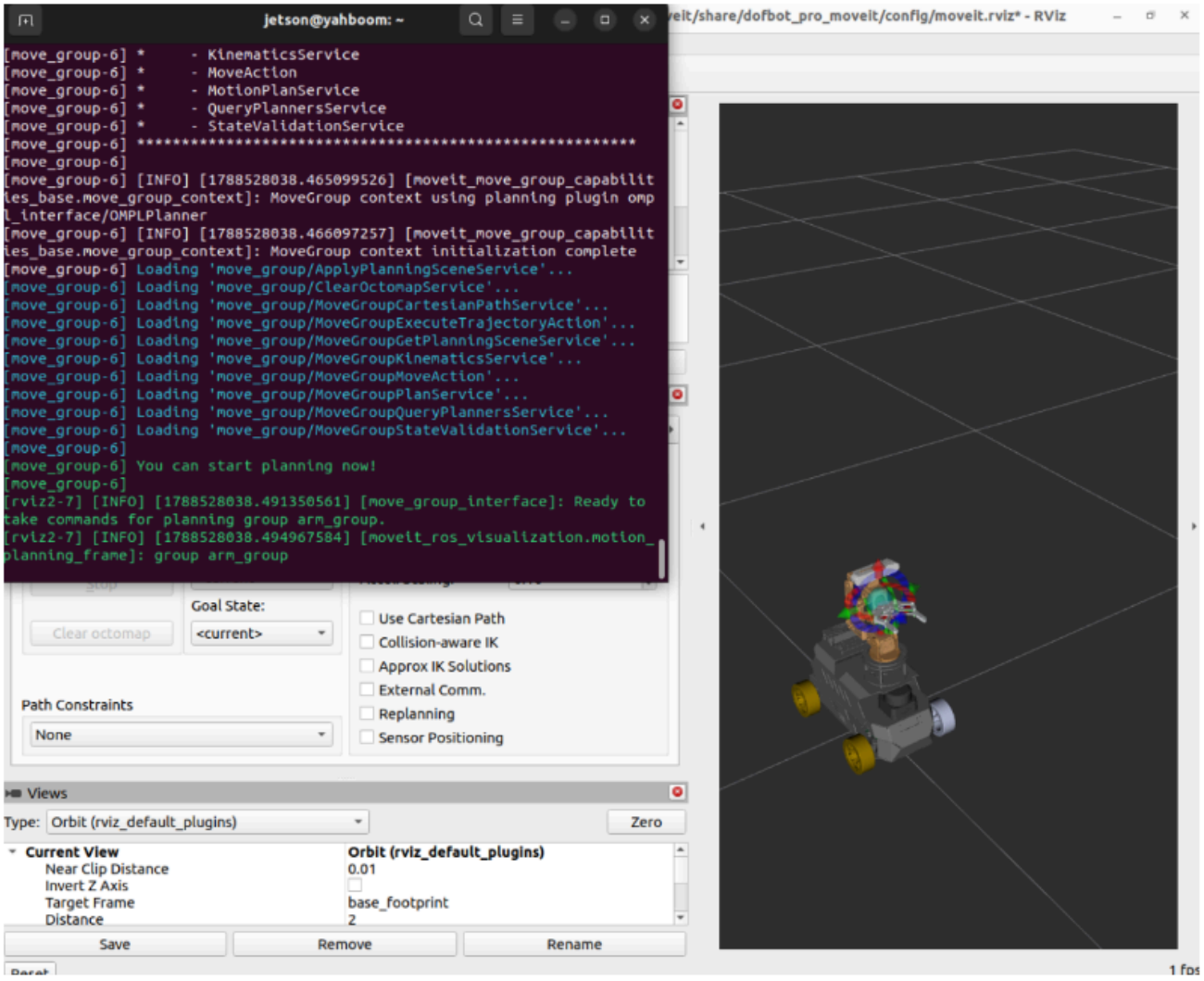
This course explains how to call the functions in the MoveIt library to implement inverse kinematics design of the robot arm. Inverse kinematics design refers to computing the required joint angles based on the desired pose of the end effector. Here, in RViz, we specify a pose for the end effector, and then let MoveIt2 plan and move to this pose.

2. Launch

Enter the following command in the terminal to launch the MoveIt2 simulation environment:

```
ros2 launch dofbot_pro_moveit demo_3d.launch.py
```

Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



If you want the simulated robot arm to be linked with the real physical robot arm (optional), open another terminal to run the real-machine driver node:

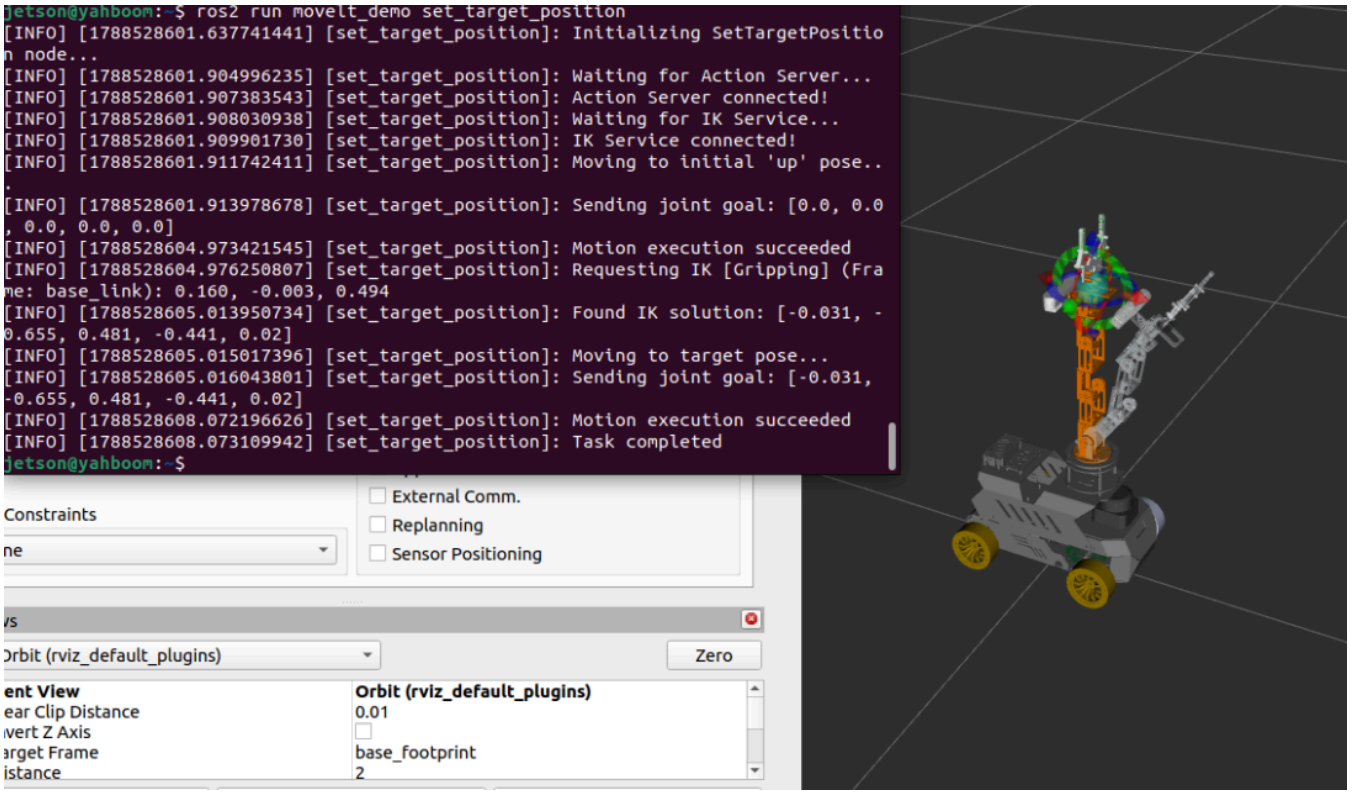
```
ros2 run moveIt_demo SimulationToMachine
```

(It is recommended not to start this node during the initial test. First verify that the obstacle-avoidance trajectory is correct in the pure RViz simulation environment before connecting to the real machine).

Then enter the following command in another terminal to launch the inverse kinematics spatial pose planning program:

```
ros2 run moveIt_demo set_target_position
```

After the program runs, the robot arm in RViz (as well as the physical robot arm) will automatically invoke the MoveIt kinematics inverse-solution solver, smoothly plan and move to the specified end-effector spatial pose, as shown in the figure below:



3. Core Code Analysis

Program source code path:
`~/yahboomcar_ros2_ws/dofbot_pro_ws/src/moveIt_demo/moveIt_demo/set_target_position.py`

3.1 Inverse Kinematics Principles and Service Client Initialization

Inverse Kinematics (IK) refers to reversely deriving and computing each joint rotation angle that satisfies the given target coordinates (x, y, z) and the orientation quaternion (q_x, q_y, q_z, q_w) of the robot arm's end effector (e.g., the gripper `Gripping`) in Cartesian 3D space.

The program creates two core interfaces:

1. `/compute_ik` **service client**: Calls MoveIt's internal KDL/IKFast solver to compute the feasible joint angles corresponding to the spatial pose;
2. `FollowJointTrajectory` **Action client**: Sends the solved joint angles to the controller to execute the physical/simulated trajectory.

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from control_msgs.action import FollowJointTrajectory
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
from builtin_interfaces.msg import Duration
from geometry_msgs.msg import PoseStamped
from moveit_msgs.srv import GetPositionIK
from moveit_msgs.msg import PositionIKRequest

class SetTargetPosition(Node):
    def __init__(self):
        super().__init__('set_target_position')
        self.get_logger().info("Initializing SetTargetPosition node...")

        # 1. Trajectory execution Action client
        self._action_client = ActionClient(
            self,
            FollowJointTrajectory,
            '/arm_group_controller/follow_joint_trajectory'
        )

        # 2. Kinematics inverse-solution solver service client
        self._ik_client = self.create_client(GetPositionIK, '/compute_ik')

        self.joint_names = [
            "arm1_Joint", "arm2_Joint", "arm3_Joint",
            "arm4_Joint", "arm5_Joint"
        ]

        self._action_client.wait_for_server()
        while not self._ik_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info("Waiting for the /compute_ik service to come online...")
```

3.2 Inverse-Solution Computing and Target Trajectory Execution

Construct a `PositionIKRequest` request, specifying the end link as `Gripping` and the target reference frame as `base_link`:

```
    def get_ik_solution(self, target_pose, frame_id="base_link", ik_link_name="Gripping"):
        """Call the /compute_ik service to compute the inverse kinematics joint angles"""
        request = GetPositionIK.Request()
        request.ik_request.group_name = "arm_group"
        request.ik_request.ik_link_name = ik_link_name
        request.ik_request.pose_stamped = target_pose
        request.ik_request.pose_stamped.header.frame_id = frame_id
        request.ik_request.timeout = Duration(sec=0, nanosec=500000000)  # 500ms timeout
        request.ik_request.avoid_collisions = False

        request.ik_request.robot_state.joint_state.name = self.joint_names
        request.ik_request.robot_state.joint_state.position = [0.0] * 5
```

```
        future = self._ik_client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        response = future.result()

        if response.error_code.val == response.error_code.SUCCESS:
            solution = response.solution.joint_state
            # Extract the joint angles of axes 1-5
            joint_positions = [
                solution.position[solution.name.index(name)]
                for name in self.joint_names
            ]
            return joint_positions
        else:
            self.get_logger().error(f"IK computation failed, error code:
{response.error_code.val}")
            return None

    def execute_ik_motion(self):
        # Set the target 3D coordinates and the quaternion orientation
        target_pose = PoseStamped()
        target_pose.pose.position.x = 0.10755
        target_pose.pose.position.y = 0.0
        target_pose.pose.position.z = 0.400775
        target_pose.pose.orientation.w = 1.0

        joint_goals = self.get_ik_solution(target_pose)
        if joint_goals:
            self.move_to_joints(joint_goals, move_time=3.0)
```